

Báo cáo rà soát test code

<!-- framework: Postman script | run_id: run_20260617_115958_012285 -->

Framework: Postman script | Run ID: run_20260617_115958_012285

Tóm tắt kỹ thuật

- Framework: Postman (Newman) script
- Kết quả chạy: Tất cả các test đều pass (passed: True) – không có lỗi runtime, không có vấn đề về cú pháp hay import.
- Coverage: Không hỗ trợ (coverage_supported: False).
- Issue type: none – không có lỗi nghiêm trọng được báo cáo.
- Artifact được xem xét: Bộ sưu tập Postman order-pricing.postman_collection.json (cũng là mã kiểm thử đã sinh).
- Artifact nguồn: Yêu cầu và kế hoạch test được cung cấp ở dạng JSON.

Lỗi nghiêm trọng

- Không phát hiện.

Test còn thiếu

- TC-006 – Subtotal = 0 (boundary)
- Bằng chứng: Kế hoạch test có TC-006, nhưng trong bộ sưu tập Postman không có test tương ứng.
- Ảnh hưởng: Không kiểm chứng hành vi khi subtotal bằng 0, mặc dù quy tắc “subtotal must be a non-negative number” yêu cầu.
- Cách sửa: Thêm một request mới với body { "subtotal":0, "weightKg":2, "destination":"domestic", "customerTier":"standard", "fragile":false } và các assertions kiểm tra status 200, diagnostics.discount == 0, diagnostics.shipping đúng.
- TC-007 & TC-008 – Clarification tests
- Bằng chứng: Các test này chỉ là yêu cầu làm rõ, không có trong collection.
- Ảnh hưởng: Thiếu kiểm chứng schema JSON và định dạng thông báo lỗi, gây rủi ro khi API thay đổi.
- Cách sửa: Thêm test (có thể là “manual” hoặc “documentation”) để xác nhận schema và format lỗi, hoặc ghi chú trong tài liệu.
- Kiểm thử bảo mật / xác thực
- Bằng chứng: Yêu cầu không đề cập tới authentication, nhưng không có bất kỳ test nào kiểm tra token, quyền truy cập.
- Ảnh hưởng: Nếu API yêu cầu auth, các test hiện tại sẽ không phát hiện lỗi bảo mật.
- Cách sửa: Thêm test với header Authorization (valid/invalid) và kiểm tra phản hồi 401/403.

Assertion yếu

- TC-001 – Kiểm tra total
- Bằng chứng: Assertion pm.expect(body.total).to.eql(100.5); nhưng yêu cầu không đề cập

tới trường total; chỉ yêu cầu các trường trong diagnostics.

- Ảnh hưởng: Assertion này có thể trở thành “always-true” nếu API luôn trả về giá trị cố định; không phản ánh đúng yêu cầu.
- Cách sửa: Loại bỏ hoặc thay thế bằng kiểm tra `body.diagnostics.totalBeforeCredit` (đã có) và tính toán total dựa trên các trường đã xác nhận.

- TC-003 – Kiểm tra “total equals domestic base shipping”
- Bằng chứng: `Assertion pm.expect(body.total).to.eql(5)`; dựa trên giả định “base shipping = 5” mà không có nguồn tham chiếu trong yêu cầu.
- Ảnh hưởng: Nếu quy tắc tính phí thay đổi, test sẽ sai mà không nhận ra lỗi logic.
- Cách sửa: Tham chiếu rõ ràng vào quy tắc tính phí trong tài liệu hoặc kiểm tra các trường `diagnostics.shipping` và `diagnostics.discount` thay vì giá trị cứng.

- TC-002, TC-004, TC-005 – Kiểm tra thông báo lỗi
- Bằng chứng: `pm.expect(body.error).to.include("subtotal")` (tương tự cho `destination`, `storeCredit`). Yêu cầu không cung cấp định dạng lỗi cụ thể.
- Ảnh hưởng: Nếu API trả về cấu trúc lỗi khác (ví dụ: `message` thay vì `error`), các test sẽ thất bại mà không phải do lỗi nghiệp vụ.
- Cách sửa: Thêm schema validation (`pm.response.to.have.jsonSchema(schema)`) hoặc kiểm tra cả `error` và `message` tùy theo spec.

Rủi ro maintainability

- Hard-coded giá trị (total: 100.5, total: 5) khiến test dễ “break” khi logic thay đổi.
- Thiếu schema validation – không có kiểm tra JSON schema cho phản hồi thành công, dẫn tới việc thay đổi cấu trúc response có thể không được phát hiện.
- Không có mock/stub – các test phụ thuộc vào môi trường thực (`http://localhost:8000`). Nếu môi trường không sẵn sàng, toàn bộ suite sẽ thất bại (flaky).
- Không có kiểm tra thời gian phản hồi – mặc dù không yêu cầu, nhưng thiếu giới hạn thời gian có thể gây lỗi khi API chậm.

Gợi ý sửa ngay

1. Thêm các test còn thiếu (TC-006, TC-007, TC-008) để bao phủ các boundary và yêu cầu tài liệu.
2. Thay thế các giá trị cứng trong assertions bằng kiểm tra dựa trên quy tắc nghiệp vụ hoặc bằng schema validation.
3. Áp dụng JSON schema validation cho cả phản hồi thành công và lỗi, ví dụ:

```
const schemaSuccess = { /* định nghĩa fields cần có */ };  
pm.response.to.have.jsonSchema(schemaSuccess);
```
4. Tách môi trường: sử dụng biến môi trường `base_url` được cấu hình trong CI, và thêm pre-request script để kiểm tra server sẵn sàng trước khi chạy.
5. Bổ sung test xác thực (Authorization header) nếu API yêu cầu, để phát hiện sớm các lỗi bảo mật.
6. Cập nhật mô tả lỗi: nếu API trả về `message` thay vì `error`, điều chỉnh các assertions cho phù hợp và ghi chú trong tài liệu.
